Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 75%. We keep your highest score.

Next item →

1. There is a flights.csv file containing the following content. 1 / 1 point

```
Airline Number,Departure,Destination,Date,Price
1,A1234,Philadelphia,San Diego,2020-02-14,500
2,A12345,Philadelphia,Los Angeles,2020-02-14,1000
3,A123456,Philadelphia,New York,2020-02-14,1000
...
```

You want to find all the flights that departed from Philadelphia and arrived in Los Angeles, for a price that is less than $1200. You already have the following script.

```
import csv
flights = []
with open('flights.csv', 'r') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        flights.append(row)


```

Which of the scripts below is true for getting the information mentioned above?

```
filtered_flights = [row in flights if row["Departure"] == Philadelphia and row["Destination"] == Los Angeles and row["Price"] < 1200)]

```

```
filtered_flights = [row for row in flights if row["Departure"] == Philadelphia and row["Destination"] == Los Angeles and row["Price"] < 1200)]

```

```
filtered_flights = [row in flights if row["Departure"] == 'Philadelphia' and row["Destination"] == 'Los Angeles' and int(row["Price"]) < 1200)]

```

```
filtered_flights = [row for row in flights if row["Departure"] == 'Philadelphia' and row["Destination"] == 'Los Angeles' and int(row["Price"]) < 1200)]

```

Nice work

The correct script for finding all the flights that departed from Philadelphia and arrived in Los Angeles, for a price that is smaller than $1200 is:

```
filtered_flights = [row for row in flights if row["Departure"] == 'Philadelphia' and row["Destination"] == 'Los Angeles' and int(row["Price"]) < 1200)]

```

2. What is the result of the following script? 1 / 1 point

```
f = lambda x: x * 4
print(f(2))

```

2222

8

16

Error

Nice work

The script is equivalent to the following script:

```
def f(x):
    return x * 4
f(2)


```

Thus, the result is 8.

3. There is a MOOC.csv file containing the following content. 1 / 1 point

```
id,name,height,weight,age
1,Melanie,72,130.6,30
2,Keenan,84,197.6,31
3,Tom,78,180.0,27
...
```

You want to get the following filtered information for the students.

```
import csv
students = []
with open('MOOC.csv', 'r') as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        students.append(row)
filtered_students = [row for row in students if int(calculate(row["weight"], row["height"])) < 2]

```

The following script is developed by another student who wants to test whether it can catch the ValueError, because there are a lot of rows in MOOC.csv file and they may contain unexpected values. Which of the statements below is true for the following script?

```
def calculate(str1, str2):
    calculated_number = 0
    num1 = float(str1)
    num2 = float(str2)
    try:
        calculated_number = num1 / num2
    except ValueError:
        print ('Error!')
    return calculated_number
print(calculate('one', 'zero'))

```

The script will cause a ValueError because the code that tries to catch the error is incorrect

The script will print 'Error!'

The script will print 'zero'

The script will print '0.0'

Nice work

This is true. The following script is the correct way to catch the error.

```
def calculate(str1, str2):
    calculated_number = 0
    try:
        num1 = float(str1)
        num2 = float(str2)
        calculated_number = num1 / num2
    except ValueError:
        print ('Error!')
    return calculated_number
print(calculate('one', 'zero'))
```

4. We have the following information about students. 1 / 1 point

```
student_info = [{'first name':'Caroline', 'last name':'Harrison', 'birthdate':'1997-03-02'}, {'first name':'Rachel', 'last name':'Reynolds', 'birthdate':'1996-01-07'}, {'first name':'Olivia', 'last name':'James', 'birthdate':'1998-05-09'}]
```

Which of the following code snippets is correct to sort the given student_info based on last name?

```
student_info.sort()

```

```
student_info.sort(student_info['last name'])

```

```
student_info.sort(key=lambda x: x['last name'])

```

None of the above

Nice work

The correct code snippet for sorting the given student_info based on last name is:

```
student_info.sort(key=lambda x: x['last name'])

```

5. We have the following information about students. 1 / 1 point

```
student_info = [{'first name':'Caroline', 'last name':'Harrison', 'birthdate':'1997-03-02'}, {'first name':'Rachel', 'last name':'Reynolds', 'birthdate':'1996-01-07'}, {'first name':'Olivia', 'last name':'James', 'birthdate':'1998-05-09'}]
```

Which of the following code snippets is correct to get the information for the youngest student. (The result should be returned as a dictionary.)

```
youngest_student_info = max(key=lambda student_info: student_info['birthdate'])

```

```
youngest_student_info = max(student_info.sort(student_info['birthdate']))

```

```
youngest_student_info = max(student_info['birthdate'])

```

```
youngest_student_info = max(student_info, key=lambda x: x['birthdate'])

```

Nice work

The correct code snippet for getting the information for the youngest student is:

```
youngest_student_info = max(student_info, key=lambda x: x['birthdate'])
```